Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 34

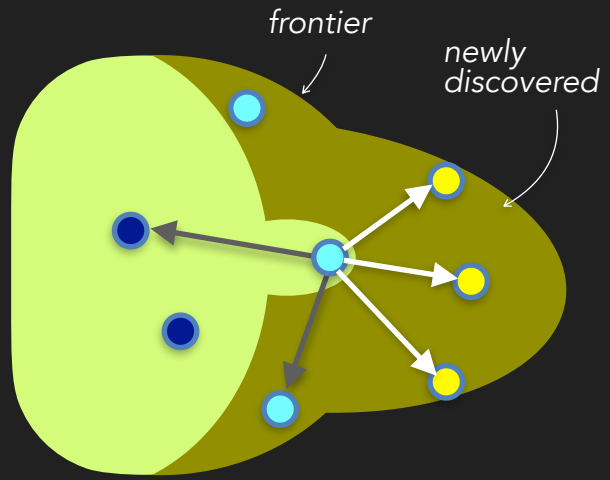
Graphs

Depth-First Search

Exploring a Graph: Template

- Recall the template for graph exploration:

```
// discovered[v] initially false for all v
frontier.insert(s); discovered[s] = true;
while (!frontier.empty()) {
  u = frontier.extract();
  for ( v : graph.neighbours(u) ) {
    if(discovered[v]) continue;
    discovered[v] = true;
    frontier.insert(v);
  }
}
```



- Marking on discovery prevents inserting a node twice into the frontier

Exploring a Graph: Template 2

- Can instead just prevent the same node from being *processed* twice

```
// activated[v] initially false for all v
frontier.insert(s);
while (!frontier.empty()) {
  u = frontier.extract();
  if(activated[u]) continue;
  activated[u] = true;
  for ( v : graph.neighbours(u) )
    frontier.insert(v);
}
```

O(|E|) iterations instead of O(|V|).
Still total time is O(|E|)

- Allows a node to get inserted into the frontier multiple times (as much as its incoming degree), possibly allowing it to be activated earlier
  - Not relevant to BFS, where first insertion will determine the extraction.
    Nor in a tree, where a node has in-degree 1.

Depth-First Search (DFS)

- DFS: Derived from our Template 2 (mark-on-activation), if frontier is implemented as a LIFO structure (stack)
- We have already seen it!
  - For (binary search) trees, with root as the source node
  - Pre-order, post-order and in-order traversal, all used DFS (but printed out the node at different points)
- What is new?
  - A graph is more general than a tree. In particular, multiple paths between a pair of nodes, or cycles.

Standard DFS

- Standard DFS is a recursive process

```
DFS(u) {
  activated[u] = true;
  for ( v : graph.neighbours(u) ) {
    if(activated[v]) continue;
    DFS(v);
  }
}
```

- In practice, for efficiency and for avoiding stack overflow on "deep" graphs, implemented without recursion, using an explicit stack
- This version behaves identically (in the order in which nodes are visited) to our mark-on-activation template, but the stack contents are slightly different

Standard DFS vs. DFS From Template 2

```
DFS(u) {
  if(activated[u]) return;
  activated[u] = true;
  for ( v : graph.neighbours(u) )
    DFS(v);
}
```

Recursive DFS

- u stays on in the (call-)stack, until all its neighbours are popped from the stack

```
frontier.insert(s);
while (!frontier.empty()) {
  u = frontier.extract();
  if(activated[u]) continue;
  activated[u] = true;
  for ( v : graph.neighbours(u) )
    frontier.insert(v);
}
```

Template DFS

- u popped from the stack, and all its neighbours pushed onto the stack.

- Say u has children $[v_1 \ldots v_d]$, and stack has (H,u) with history H

  In template: $(H,u) \rightarrow (H,v_d \ldots v_1) \dashrightarrow (H,v_d \ldots v_2) \dashrightarrow (H,v_d) \rightarrow (H)$

  In recursive: $(H,u) \rightarrow (H,(u,1),v_1) \dashrightarrow (H,(u,2),v_2) \dashrightarrow (H,(u,d)) \rightarrow (H)$ (stack frame has for-loop counter)

Equivalent executions with stack entries represented differently

Standard DFS

- Standard DFS obtained by a further minor optimisation

```
DFS(u) {
  if(activated[u]) return;
  activated[u] = true;
  for ( v : graph.neighbours(u) )
    DFS(v);
}
```

```
DFS(u) {
  activated[u] = true;
  for ( v : graph.neighbours(u) ) {
    if(activated[v]) continue;
    DFS(v);
  }
}
```

- Do not make the recursive call if it is just going to return
  - Still O(|E|) time, but the number of function calls is O(min(|V|,|E|)) instead of O(|E|), as DFS is called on each reachable node exactly once
- For further efficiency/avoiding stack overflow often implemented without recursion, using an explicit stack

Active Nodes in DFS

- To study DFS (recursive version) consider the "active lifetime" of a node u, from the point when DFS(u) is invoked till it returns
  - Define activation time A[u] and exit time X[u]

1. At any point all active nodes lie in a path from the source to the last activated
   - These are the nodes in the call-stack
2. If v reachable from u, and not activated when u activated (i.e., A[u] < A[v]) then v gets activated while u is still active. Further, then v must exit while u is still active (i.e., A[u] < A[v] < X[v] < X[u])
   - If v not reachable from u, and A[u] < A[v], then A[u] < X[u] < A[v] < X[v]

Cycle Detection

- A cycle: $u_0,\ldots,u_d$ s.t. $(u_{i-1},u_i) \in E$ for $i=1,\ldots,d$, and u_i's distinct except $u_0=u_d$
- DFS (recursive version) can be easily adapted to check if a graph has a cycle.
- W.l.o.g, assume there is a source node from which all nodes are reachable
  - If necessary, add a new source node with an edge to every node: this cannot add or remove cycles. $|E_{new}| = |V| + |E|$, $|V_{new}| = |V|+1$.
- To detect cycles, run the following variant of DFS from this all-reaching source

```
DFS(u) {  // uses external boolean vectors activated and active
  active[u] = true; activated[u] = true;
  for ( v : graph.neighbours(u) ) {
    if(active[v]) report_cycle();
    if(activated[v]) continue;
    DFS(v);
  }
  active[u] = false;
}
```

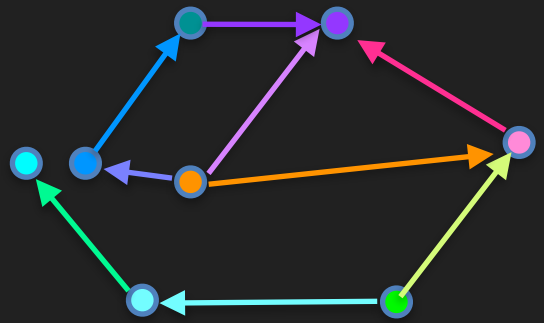
Saw active v from active u

Cycle Detection

- Claim: If it reports a cycle, there is a cycle
  - Reports a cycle: saw an edge (u,v) when v is active
    - u is the last activated node. So there is a path s ⇢ v ⇢ u (by 1️⃣). Edge (u,v) completes the cycle v ⇢ u → v
- Claim: If there is a cycle C (reachable), it will be found
  - Let v be the first node in C that is activated. Let u be s.t. C is v ⇢ u → v.
    - If u=v (C is just a self-loop), DFS(v) will see the edge to active v.
    - Suppose u≠v. Then u not activated when v activated. So DFS(v) will activate u eventually (by 2️⃣); then DFS(u) will see active v and report cycle.

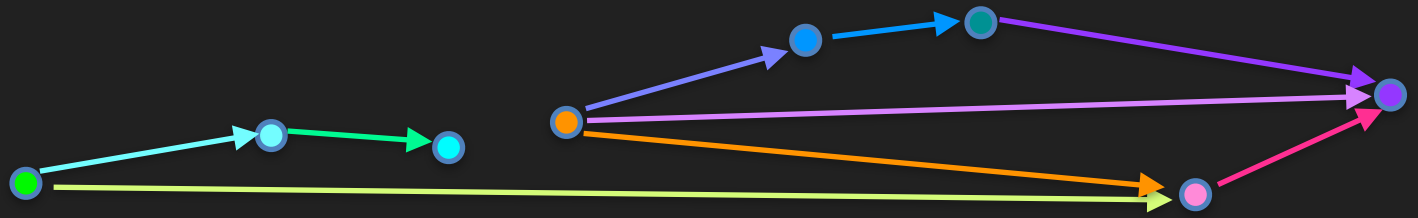
Topological Sorting

- A topological sort is an ordering of all the nodes in a directed graph such that all the edges point forward



Topological Sorting

- A topological sort is an ordering of all the nodes in a directed graph such that all the edges point forward



- Not possible if there is a cycle
- Turns out, otherwise, always possible!
  - A directed graph without cycles is called a Directed Acyclic Graph (DAG)

Topological Sorting

- Claim: Every DAG can be topologically sorted
  - In a DAG, the cycle detection DFS never reports a cycle
  - Claim: In a DAG, for any edge (u,v) , X[v] < X[u]. Consider cases:
    - A[v] < A[u]: Can't have A[v] < A[u] < X[v], because then cycle reported on exploring edge (u,v). So X[v] < A[u] and hence X[v] < X[u]
    - A[u] < A[v]: Since v reachable from u, by 2️⃣, A[u] < A[v] < X[v] < X[u]
  - Ordering nodes by decreasing X time-stamp gives a topological sorting!
- We may have added an artificial source s for DFS. X[s] is the maximum, and hence the first node in the topological ordering. Drop it if artificial node.

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

Honour Code: Don't leave the lecture after marking attendance!

If you can't attend the whole lecture don't mark attendance!

If you are not in the classroom, a report will be sent to DADAC!

